BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 11
Functions

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Function Basics	-	1	4-12
3	Function Prototypes	-	2	13-16
4	Summary	-	-	17
5	Terminal Questions	-	-	18
6	Answers To Self Assessment Questions	-	-	18
7	Answers To Terminal Questions	-	-	18-19
8	Exercises	-	-	19

1. INTRODUCTION

In the preceding module, we delved into the realm of C arrays. Now, as we transition into this new unit, our focus shifts towards understanding the concept of a function.A function can be likened to a "black box," encapsulating a specific portion of our program. It serves as a compartmentalized entity, embodying certain key properties:

Firstly, a function is designed to execute a well-defined task, serving a specific purpose within the program's logic. This task is intended to be reusable, not only within the current program but potentially across various other programs as well, thus promoting code reusability and modularity. In essence, functions provide a structured approach to organizing and modularizing code, enhancing readability, maintainability, and the overall efficiency of software development.

Functions are probably the most important weapon in our battle against software complexity. You'll want to learn when it's appropriate to break processing out into functions (and also when it's not), and how to set up function interfaces to best achieve the qualities mentioned above: reusability, information hiding, clarity, and maintainability.

1.1. Objectives:

At studying this unit, you should be able to:

- *explain the importance of functions*
- *implement the concepts of formal arguments and actual arguments*
- *explain function declaration(function prototypes) and function definition*

2. FUNCTION BASICS

A function is a self-contained program segment that carries out some specific, well-defined task. Every C program contains one or more functions. One of these functions must be called main. Program execution will always begin by carrying out the instructions in main. Additional functions will be subordinate to main, and perhaps to one another.

So what defines a function? It has a name that you call it by, and a list of zero or more arguments or parameters. Parameters (also called formal parameters) or arguments are the special identifiers through which information can be passed to the function. A function has a body containing the actual instructions (statements) for carrying out the task the function is supposed to perform; and it may give you back a return value, of a particular type.

In general terms, the first line can be written as

data-type name(data-type parameter 1, data-type parameter 2, ..., data-type parameter n)

Example 11.1: Here is a very simple function, which accepts one argument, multiplies it by 4, and hands that value back.

```
int multbyfour(int x)
{
int retval;
retval = x * 4;
return retval;
}
```

On the first line we see the return type of the function (int), the name of the function (multbyfour), and a list of the function's arguments, enclosed in parentheses. Each argument has both a name and a type; **multbyfour** accepts one argument, of type **int**, named x. The name x is arbitrary, and is used only within the definition of **multbyfour**. The caller of this function only needs to know that a single argument of type **int** is expected; the caller does

not need to know what name the function will use internally to refer to that argument. (In particular, the caller does not have to pass the value of a variable named x.)

Next we see, surrounded by the familiar braces, the body of the function itself. This function consists of one declaration (of a local variable *retval*) and two statements. The first statement is a conventional expression statement, which computes and assigns a value to *retval*, and the second statement is a *return* statement, which causes the function to return to its caller, and also specifies the value which the function returns to its caller.

In general term, a *return* statement is written as

return expression

The *return* statement can return the value of any expression, so we don't really need the local *retval* variable; this function can also be written as

```
int multbyfour(int x)
{
return x * 4;
}
```

How do we call a function? We've been doing so informally since day one, but now we have a chance to call one that we've written, in full detail. The arguments in the function call are referred to as *actual arguments or actual parameters,* in contrast to the formal arguments that appear in the first line of the function definition.

Here is a tiny skeletal program to call multbyfour:

```
#include <stdio.h>

extern int multbyfour(int);

int main()

{

int i, j;

i = 5;

j = multbyfour(i);

printf("%d\n", j);

return 0;

}
```

This looks much like our other test programs, with the exception of the new line

extern int multbyfour(int);

This is an *external function prototype declaration*. It is an external declaration, in that it declares something which is defined somewhere else. (We've already seen the defining instance of the function **multbyfour**, but may be the compiler hasn't seen it yet.) The function prototype declaration contains the three pieces of information about the function that a caller needs to know: the function's name, return type, and argument type(s). Since we don't care what name the **multbyfour** function will use to refer to its first argument, we don't need to mention it. (On the other hand, if a function takes several arguments, giving them names in the prototype may make it easier to remember which is which, so names may optionally be used in function prototype declarations.) Finally, to remind us that this is an external declaration and not a defining instance, the prototype is preceded by the keyword **extern**.

The presence of the function prototype declaration lets the compiler know that we intend to call this function, **multbyfour**. The information in the prototype lets the compiler generate the correct code for calling the function, and also enables the compiler to check up on our code (by making sure, for example, that we pass the correct number of arguments to each function we call).

Down in the body of main, the action of the function call should be obvious:

the line

```
j = multbyfour(i);
```

calls B, passing it the value of i as its argument. When **multbyfour** returns, the return value is assigned to the variable j. (Notice that the value of main's local variable i will become the value of **multbyfour's** parameter x; this is absolutely not a problem, and is a normal sort of affair.)

This example is written out in "longhand,'' to make each step equivalent. The variable i isn't really needed, since we could just as well call

```
j = multbyfour(5);
```

And the variable j isn't really needed, either, since we could just as well call

```
printf("%d\n", multbyfour(5));
```

Here, the call to **multbyfour** is a subexpression which serves as the second argument to **printf**. The value returned by **multbyfour** is passed immediately to **printf**. (Here, as in general, we see the flexibility and generality of expressions in C. An argument passed to a function may be an arbitrarily complex subexpression, and a function call is itself an expression which may be embedded as a subexpression within arbitrarily complicated surrounding expressions.)

We should say a little more about the mechanism by which an argument is passed down from a caller into a function. Formally, C is *call by value*, which means that a function receives *copies* of the values of its arguments. We can illustrate this with an example. Suppose, in our

implementation of **multbyfour**, we had gotten rid of the unnecessary **retval** variable like this:

```
int multbyfour(int x)
{
x = x * 4;
return x;
}
```

We might wonder, if we wrote it this way, what would happen to the value of the variable i when we called

```
j = multbyfour(i);
```

When our implementation of **multbyfour** changes the value of x, does that change the value of i up in the caller? The answer is no. x receives a copy of i's value, so when we change x we don't change i.

However, there is an exception to this rule. When the argument you pass to a function is not a single variable, but is rather an array, the function does *not* receive a copy of the array, and it therefore *can* modify the array in the caller. The reason is that it might be too expensive to copy the entire array, and furthermore, it can be useful for the function to write into the caller's array, as a way of handing back more data than would fit in the function's single return value. We will discuss more about passing arrays as arguments to a function later.

There may be several different calls to the same function from various places within a program. The actual arguments may differ from one function call to another. Within each function call, however, the actual arguments must correspond to the formal arguments in the function definition; i.e the actual arguments must match in number and type with the corresponding formal arguments.

Program 11.1: A program to find the largest of three integers

```
#include<stdio.h>
main()
{
int x, y, z, w;
/* read the integers */
int max(int, int);
printf("\nx= ");
scanf("%d", &x);
printf("\ny= ");
scanf("%d", &y);
printf("\nz= ");
scanf("%d", &z);
/* Calculate and display the maximum value */
w= max(x,y);
printf("\n\nmaximum=%d\n", max(z,w));
}
int max(int a, int b)
{
int c;
c=(a>=b)?a:b;
return c;
}
```

Please execute this program and observe the result.

Function calls can span several levels within a program; function A can call function B, which can call function C and so on.

Program 11.2: Program to check whether a given integer is a perfect square or not.

```
#include<stdio.h>
main()
{
        int psquare(int);
        int num;
        printf(" Enter the number:");
          scanf("%d", &num);
          if(psquare(num))     /* main() calls the function psquare() */
          {
                printf("%d is a perfect square\n");
          else
                printf("%d is not a perfect square\n");
          }
  }
  int psquare(int x)
  {
  int positive(int);
  float sqr;
  if(positive(x))  /*  psquare()  in  turn  calls  the  function
  positive() */ {
```

```
        sqr=sqrt(x));
        if(sqr-int(sqr))==0)
            return 1;
        else
            return 0;
    }
    int positive(int m)
    {
        if(m>0)
            return 1;
        else return 0;
    }
```

Execute the above program and observe the result.

In the above program the **main** function calls the function psquare() and it in turn calls the function positive() to check whether the number to be checked for perfect square is a positive or not before checking.

The **return** statement can be absent altogether from a function definition, though this is generally regarded as a poor programming practice. If a function reaches end of the block without encountering a return statement, control simply reverts back to the calling portion of the program without returning any information. Using an empty return statement(without the accompanying expressions) is recommended.

Example 11.2: The following function accepts two integers and determines the larger one, which is then written out. The function doesn't return any information to the calling program.

```
void max(int x, int y)
{
        int m;
        m=(x>=y)?x:y;
        printf(" The larger integer is=%d\n", m);
        return;
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. The function main() is optional in a C program. (True/False)
2. If the function is defined elsewhere (not in the same program where it is called), the function prototype must be preceded by the keyword _____.
3. The arguments that appear in function definition are called _____ arguments whereas the arguments that appear when the function is called are the _____ arguments.
4. The return data type, function name and the list of formal parameters enclosed in brackets are separated by _____.

3. FUNCTION PROTOTYPES

In modern C programming, it is considered good practice to use prototype declarations for all functions that you call. As we mentioned, these prototypes help to ensure that the compiler can generate correct code for calling the functions, as well as allowing the compiler to catch certain mistakes you might make.

In general terms, a function prototype can be written

as *data-type name(type1, type2, ..., type n)* Examples:

int sample(int, int) or int sample(int a, int b);

float fun(int, float) or float fun(int a, float b);

void demo(void); Here **void** indicates function neither return any value to the caller nor it has any arguments.

If you write the function definition after the definition of its caller function, then the prototype is required in the caller, but the prototype is optional if you write the definition of the function before the definition of the caller function. But it is good programming practice to include the function prototype wherever it is defined.

If prototypes are a good idea, and if we're going to get in the habit of writing function prototype declarations for functions we call that we've written (such as multbyfour), what happens for library functions such as **printf?** Where are their prototypes? The answer is in that boilerplate line

```
#include <stdio.h>
```

we've been including at the top of all of our programs. stdio.h is conceptually a file full of external declarations and other information pertaining to the "Standard I/O'' library functions, including printf. The #include directive arranges all of the declarations within stdio.h that are considered by the compiler, rather as if we'd typed them all in ourselves. Somewhere within these declarations is an external function prototype declaration for printf, which satisfies the rule that there should be a prototype for each function we call. (For

other standard library functions we call, there will be other “header files'' to include.) Finally, one more thing about external function prototype declarations: we've said that the distinction between external declarations and defining instances of normal variables hinges on the presence or absence of the keyword extern. The situation is a little bit different for functions. The “defining instance'' of a function is the function, including its body (that is, the brace-enclosed list of declarations and statements implementing the function). An external declaration of a function, even without the keyword extern, looks nothing like a function declaration. Therefore, the keyword extern is optional in function prototype declarations.

If you wish, you can write

```
int multbyfour(int);
```

and this is just like an external function prototype declaration as

```
extern int multbyfour(int);
```

(In the first form, without the extern, as soon as the compiler sees the semicolon, it knows it's not going to see a function body, so the declaration can't be a definition.) You may want to stay in the habit of using extern in all external declarations, including function declarations, since “extern = external declaration'' is an easier rule to remember.

Program 11.3: Program to illustrate that the function prototype is optional in the caller function. The program is to convert a character from lowercase to uppercase.

```
#include<stdio.h>
char lower_to_upper(char ch) /* Function definition precedes main()*/
{
        char c;
        c=(ch>=’a’ && ch<=’z’) ? (‘A’+ch-‘a’):ch;
        return c;
}
```

```
main()
{
        char lower, upper;
            /* char lower_to_upper(char lower); */ /* Function prototype is optional
        here*/
        printf("Please enter a lowercase character:");
        scanf("%c", &lower);
        upper=lower_to_upper(lower);
        printf("\nThe uppercase equivalent of %c is %c\n", lower, upper);
}
```

SELF-ASSESSMENT QUESTIONS - 2

5. Function prototype is also called _____.
6. The function prototypes are optional. (True/False)
7. The function prototypes of the library functions is in the corresponding _____ files.
8. The function prototype for the function fun() called in main() below is _____ .

```
main()
    {
        double x, y, z;
        ...
        z=fun(x, y);
        ...
    }
```

4. SUMMARY

A function in programming is akin to an autonomous segment of code dedicated to executing a particular, clearly defined task. When grappling with the complexity of a program, it often stems from a lack of organization, where the code hasn't been appropriately divided into distinct functions. Essentially, a function acts as a self-contained "black box," housing a portion of our program logic. The fundamental concept behind functions lies in their ability to compartmentalize program components, thereby enhancing modularity and maintainability.

The main() function holds utmost significance in every C program, serving as the entry point for program execution. Additionally, a function prototype serves as an early declaration of the function, outlining its name, parameters, and return type, facilitating modular design and enabling the compiler to recognize function calls before their actual definitions.

INSPIRED BY LIFE

5. TERMINAL QUESTIONS

1. What is the purpose of a function prototype?
2. How do you define a function in C?
3. What is the syntax for calling a function in C?
4. Can a function return a value? If so, how?
5. How are parameters passed to a function in C?
6. What is the difference between formal parameters and actual parameters?

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. False
2. extern
3. formal, actual
4. comma
5. Function declaration
6. False
7. header
8. double fun(double, double);

7. ANSWERS TO TERMINAL QUESTIONS

1. 'void' is the keyword used to specify that the function doesn't return any value. It can also be used to specify the absence of arguments.
2. Function declaration is a direction to the compiler that what type of data is returned by the function, the function name and about the arguments where as the function definition is actually writing the body of the function along with the function header.
3. #include<stdio.h>

   main()

   {

   int n=10;

```
int fun(int n); printf("%d", fun(n));
}
int fun(int n)
{
if(n>0) return (n+fun(n-2));
}
```

4.

```
gcd(m,n)= {     m or n        if m=n
                GCD(m, m-n) if m>n
                GCD(n,m)      if m<n }
```

8. EXERCISES

1. Suppose function F1 calls function F2 within a C program. Does the order of function definitions make any difference? Explain.
2. When a program containing recursive function calls is executed, how are the local variables within the recursive function interpreted?
3. Express the following algebraic formula in a recursive form: Y = (x1+x2+…+xn)
4. Write a function that will allow a floating point number to be raised to an integer power.
5. Write a function to swap two numbers using pass by value technique. What is the drawback of the function?